



PRODUCT BLUEPRINT / SEPTEMBER 2026

OmniRoute

Unified multi-LLM workspace: product architecture, user experience, workflows, technology stack, delivery plan, and product assessment

Core promise

Ask once. Compare three leading AIs. Choose the best response. Continue normally. Switch to another AI at any moment without losing the relevant conversation context.

Prepared for Rishabh

Single-document architecture and implementation guide

Scope: API-backed providers and platform credits; consumer subscription inheritance excluded



Executive summary

OmniRoute should be built as a conversation operating layer, not as three chat windows placed beside one another. The platform owns the canonical history, files, memory, branches, cost records, and permissions. GPT, Claude, Gemini, and future providers are replaceable execution engines behind a stable provider interface.

Recommended product thesis

Your conversation stays. Your AI can change. The defensible value is cross-model continuity, response comparison, capability-aware switching, reliable usage accounting, and a growing dataset of which response users actually prefer.

Decision	Recommendation	Why it matters
Product mode	Compare 3 for important prompts; Single AI for continued work	Balances quality, speed, and inference cost.
Architecture	Modular monolith first	Fast to ship and test while preserving clear service boundaries.
Frontend	Next.js 16 App Router + TypeScript	Current full-stack React framework with strong streaming and deployment support.
Backend	NestJS 11 on Node.js 24 LTS, Fastify adapter	Structured modules, DI, validation, queues, SSE, testing, and clear interview value.
Data	PostgreSQL 18 + Redis + object storage	Relational correctness for branches and credits; fast ephemeral coordination; scalable file handling.
Transport	REST for commands; SSE for multiplexed token streams	Simple browser support, reconnect behavior, and lower operational complexity than WebSockets.
Providers	OpenAI Responses, Anthropic Messages, Gemini Interactions	Uses each provider's current recommended API surface as of September 2026.
Deployment	Vercel web + managed container API + managed Postgres/Redis	Low-ops MVP with a straightforward path to AWS and Kubernetes later.

This report is intentionally opinionated. It recommends one primary build path, records alternatives, and separates what belongs in the first useful release from what should wait until real usage justifies it.

Contents

The architecture is organized from product experience to implementation and operational reality.

1. Product definition and boundaries	4
2. User experience blueprint	6
3. End-to-end user workflows	8
4. System architecture	10
5. Provider abstraction and orchestration	12
6. Conversation branching and shared context	14
7. Data model and transactional integrity	16
8. API and streaming contract	18
9. Recommended market-aligned technology stack	19
10. Building structure and repository design	21
11. DevOps, deployment, and environments	22
12. Security, privacy, and responsible AI controls	24
13. Observability, reliability, and quality evaluation	25
14. Implementation roadmap	26
15. Testing and acceptance strategy	27
16. Product positives, negatives, and market reality	28
17. Final recommendation	30
Sources and current-technology notes	32

1. Product definition and boundaries

What OmniRoute is, what it is not, and why the distinction matters.

OmniRoute is a unified multi-model workspace. A user sends one prompt to a chosen set of providers, views each response in a coherent comparison interface, selects the preferred answer, and continues the conversation with that provider. At any later point the user can switch providers; OmniRoute reconstructs the relevant shared context so the new provider starts from the current state instead of from zero.

The platform is therefore responsible for four kinds of continuity:

- Conversation continuity: the active branch, selected responses, recent turns, summaries, and pinned instructions.
- Artifact continuity: uploaded files, generated code, images, citations, and reusable outputs.
- Economic continuity: reservations, token usage, credits, refunds, and provider-specific cost records.
- Experience continuity: a stable composer, history, keyboard behavior, streaming pattern, and error model across providers.

In scope for V1	Explicitly later
Google login, chat history, Compare 3, Single AI, streaming, response selection	Consumer ChatGPT Plus / Claude Pro / Gemini plan inheritance
One-click provider switching with shared relevant context	Autonomous AI council or debate agents
Platform credits, usage ledger, quotas, provider fallback	Machine-learning router trained on selection data
Text, files, capability registry, basic image-aware routing	Kubernetes, Kafka, service mesh, multi-region active-active
OpenAI, Anthropic, Gemini provider adapters	Every AI provider and every provider-native tool

Avoid the weak pitch

Do not present the product as an 'AI API aggregator.' Present it as a cross-model conversation workspace that removes the manual copying, re-uploading, and re-explaining users perform today.

1.1 Target users and jobs-to-be-done

User	Primary job	Why compare or switch
Developers	Debug, design, generate, and review code	A second model can challenge a failed solution or continue with a different coding strength.
Students	Understand concepts and improve answers	Different explanations expose gaps and offer better teaching styles.
Writers and marketers	Draft, rewrite, and ideate	Users choose tone and structure, then continue without copying drafts.
Researchers	Explore, synthesize, and inspect files	Different providers may offer different tools, context handling, and retrieval behavior.
Small teams	Maintain one shared AI-assisted project history	Work survives individual provider changes and can be audited centrally.

The sharpest initial wedge is developers and technical students. Their tasks are easy to demonstrate, context continuity has obvious value, and model switching can be measured by concrete outcomes such as accepted fixes, regenerated answers, and successful tests.

2. User experience blueprint

The interface should feel like one capable workspace, not three products stitched together.

The user should make only two major decisions: how many AIs to ask and which answer to continue with. Provider mechanics, API differences, retries, token accounting, and context conversion stay behind the interface unless the user asks for detail.

2.1 UX principles

Principle	Product behavior
One workspace	A single sidebar, composer, file area, history, and project memory remain stable across models.
Compare without overload	Use provider tabs or a response carousel by default; offer desktop split view as an optional comparison mode.
Selection is explicit	A clear 'Continue with this response' action creates the active branch. Never silently discard alternatives.
Switching is lightweight	Provider picker shows capability, availability, estimated credit range, and current context readiness.
Costs are understandable	Show an estimated range before sending and actual usage after completion; avoid pretending fixed credits equal fixed tokens.
Failure is partial, not total	If one provider fails, keep the other responses visible and provide per-provider retry.
Trust is inspectable	Show the active provider and model on every answer, plus a compact context handoff indicator after switching.

Mobile rule

Do not place three long answers in three narrow columns. Use tabs, horizontal swiping, response position such as 2 of 3, and a sticky selection button.

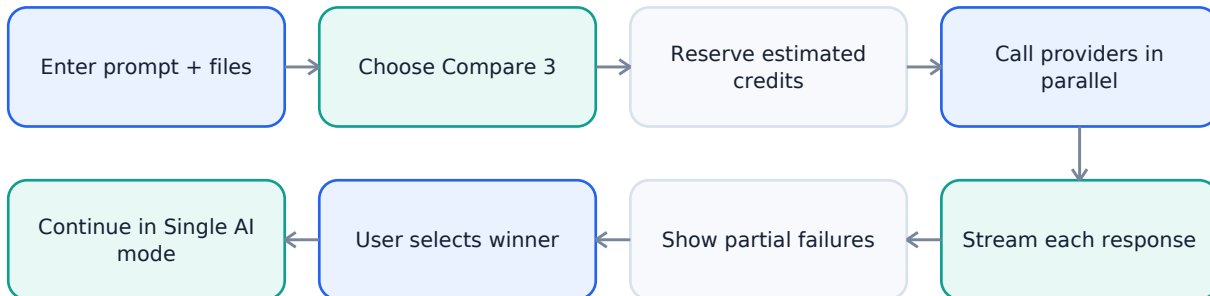
2.2 Core screens and information architecture

Screen	Essential elements	Primary success signal
Home / new chat	Composer, recent chats, Compare 3 and Single AI choices, selected providers	First useful prompt sent with minimal setup.
Comparison result	Provider tabs, independent streaming states, latency, retry, select response	User selects one answer or deliberately asks a follow-up comparison.
Active conversation	Normal chat, active model badge, switch menu, files, branch breadcrumb	User continues without thinking about context transfer.
Switch model	Capabilities, availability, cost estimate, context status	Provider changes in one click and answers coherently.
History	Chats, search, projects, active provider, branch count	Prior work is found and resumed quickly.
Usage	Credits, actual tokens, provider cost, daily limit, transaction ledger	User understands why credits changed.
Settings	Privacy, data retention, provider preferences, export/delete	User can control stored data and defaults.

3. End-to-end user workflows

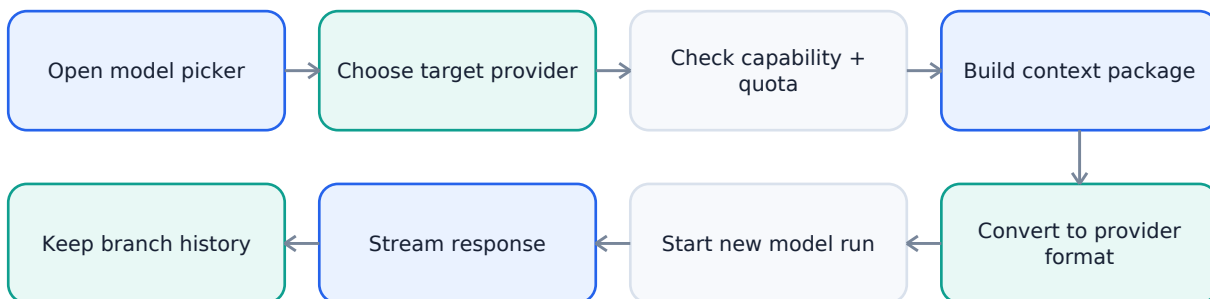
Every workflow preserves momentum and makes the active branch unambiguous.

3.1 New chat: compare, select, continue



1. User writes a prompt, attaches files if needed, and sees an estimated credit range.
2. Backend creates one user turn and three model runs under a shared request group.
3. Provider adapters execute concurrently. The UI receives normalized stream events tagged by run ID.
4. All completed candidates remain saved. The user explicitly selects one as the active response.
5. The next prompt defaults to the selected provider in Single AI mode, reducing cost and latency.

3.2 Instant provider switch



A switch does not copy every historical candidate. The context engine follows the active selected path, adds pinned memory, current artifacts, a conversation summary, relevant older turns, and the latest user request. Rejected alternatives stay stored but are excluded unless the user references them.

3.3 Provider limit or outage

Recommended recovery copy

'Claude is temporarily unavailable. Continue this conversation with GPT or Gemini. Your selected context and files will stay attached.' The system should not claim zero context loss; it should promise transfer of the relevant saved context.

- If failure happens before meaningful output, release the reserved amount for that run.
- If partial output exists, let the user keep it and charge according to actual billable usage, subject to a transparent minimum policy.
- Do not automatically submit private prompts to another provider unless the user has enabled automatic fallback.
- Record provider error category, retry-after information, and request ID without exposing secrets.

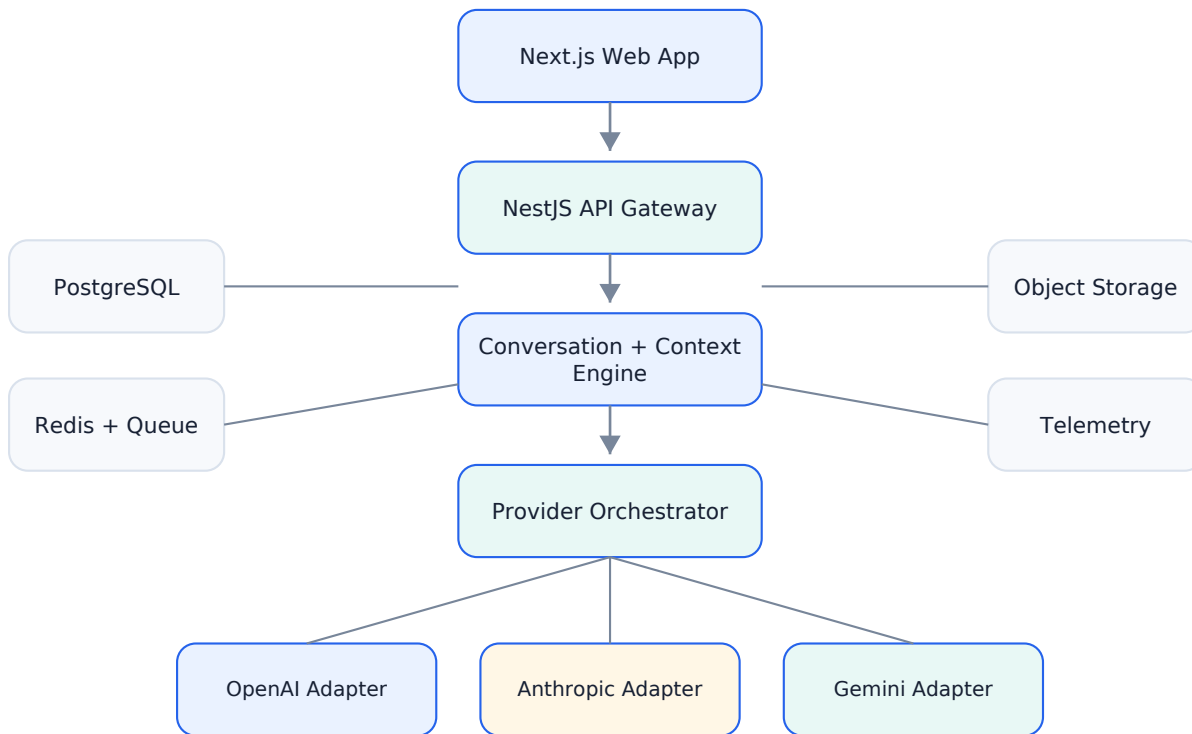
3.4 Capability-aware task switching

When the task changes from coding to image generation, web research, file analysis, or long-context review, the provider picker should rank models by capability without forcing an automatic choice. The provider registry is versioned configuration, not a permanent hard-coded comparison table.

Task	System check	UX response
Generate an image	Target model supports image generation and requested aspect ratio	Show capable providers first; explain unavailable options.
Analyze a PDF	File type, size, OCR requirement, provider upload limits	Reuse the stored file; transform or chunk only when needed.
Run code or tools	Tool allowlist, sandbox, user permission, provider tool format	Show planned action and require approval for external side effects.
Search current information	Provider has search or platform retrieval is enabled	Label sources and the provider/tool that retrieved them.
Long conversation	Estimated tokens exceed target window or cost ceiling	Summarize and retrieve relevant history before switching.

4. System architecture

Start as a modular monolith with independent provider adapters and operational boundaries.



The diagram separates ownership of state from execution of inference. PostgreSQL is the system of record. Redis accelerates ephemeral coordination but is never the only copy of a credit transaction or selected branch. Object storage holds large files and artifacts. The provider orchestrator fans out model calls and emits normalized events.

4.1 Why a modular monolith first

- One deployable backend is easier to debug, test, observe, and afford during an MVP.
- NestJS modules still create boundaries around identity, conversations, context, providers, usage, files, and analytics.
- Heavy or unreliable work moves to background workers without requiring a premature microservice program.
- Services can be extracted later from measured bottlenecks, not imagined scale.

Do not start with

Kubernetes, Kafka, event sourcing, a separate vector database, a service mesh, or an ML router. Each adds operational work without improving the first comparison-and-switch loop.

4.2 Backend modules and responsibilities

Module	Owns	Must not own
Identity	Users, OAuth identities, sessions, roles, deletion requests	Provider API keys in application logs
Conversation	Chats, turns, branches, selected responses, titles	Provider-specific request formatting
Context	Active path, summaries, pinned memory, retrieval, token budget	Credit deduction or provider billing
Provider hub	Adapters, capabilities, timeouts, retries, normalized streams	Canonical conversation state
Usage and credits	Reservations, actual usage, pricing snapshots, ledger entries	Mutable balance without transactions
Files and artifacts	Metadata, signed access, scanning, chunking, lifecycle	Large file bytes in PostgreSQL
Policy	Limits, content controls, tool permissions, retention settings	Business analytics definitions
Analytics	Selection events, switch behavior, latency, quality feedback	Raw sensitive content by default

5. Provider abstraction and orchestration

Normalize the common core without pretending every provider is identical.

Each adapter accepts a canonical request and emits canonical events. Provider-specific features remain available through capability flags and typed extension fields. This avoids a lowest-common-denominator interface while keeping the rest of the application independent from SDK churn.

```
interface AIPProvider {
  id: ProviderId;
  capabilities(): Promise<CapabilitySnapshot>;
  streamChat(request: CanonicalChatRequest): AsyncIterable<ProviderEvent>;
  cancel(runId: string): Promise<void>;
  estimate(request: CanonicalChatRequest): Promise<UsageEstimate>;
}

type ProviderEvent =
  | { type: 'run.started'; providerRequestId?: string }
  | { type: 'content.delta'; text: string }
  | { type: 'tool.call'; call: CanonicalToolCall }
  | { type: 'usage.updated'; inputTokens?: number; outputTokens?: number }
  | { type: 'run.completed'; usage: NormalizedUsage }
  | { type: 'run.failed'; error: NormalizedProviderError };
```

5.1 Parallel comparison orchestration

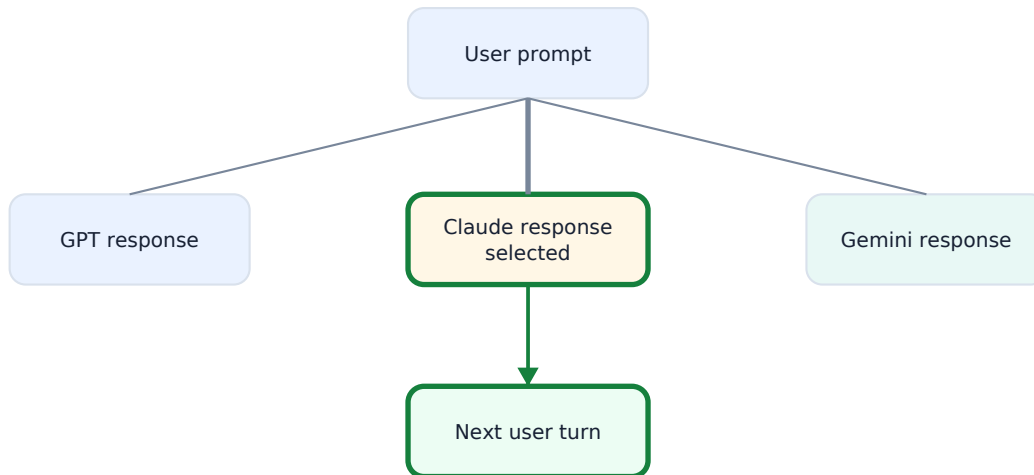
1. Validate user, workspace, provider selections, file permissions, and quota.
2. Create an idempotent request group, a user turn, and one pending model run per provider.
3. Reserve the maximum allowed credits transactionally before calling external APIs.
4. Start adapters concurrently with per-provider timeouts and independent cancellation tokens.
5. Multiplex normalized events over a single SSE connection using run IDs and monotonically increasing event IDs.
6. Persist final outputs and usage, reconcile reserved versus actual credits, and publish terminal states.
7. Allow selection after one response completes; do not require the slowest provider to finish.

5.2 Retry and idempotency policy

Failure	Retry policy	User result
Network reset before response	One bounded retry with same logical idempotency key	Continue streaming if safe; avoid duplicate candidate records.
Provider 429	Honor retry-after; no rapid loop	Show queued/retry state and let other providers finish.
Provider 5xx	Exponential backoff with jitter; circuit breaker after threshold	Offer individual retry or another provider.
Invalid request	No automatic retry	Normalize error and flag adapter/configuration issue.
Client disconnect	Continue only when background completion is enabled	Reconnect from last SSE event or fetch run state.

6. Conversation branching and shared context

The active conversation is a path through a tree, not a flat provider-owned thread.



For each user turn, OmniRoute may store multiple model responses. One response becomes active. The next user turn points to that selected response, which makes the branch explicit and reversible. If the user later chooses another candidate from an earlier turn, the application creates or activates a different path rather than rewriting history.

6.1 Context assembly algorithm

1. Resolve the active branch from the current conversation head back to the root.
2. Collect system instructions, workspace rules, pinned user memory, and current task metadata.
3. Attach the latest turns verbatim within a fixed recent-history budget.
4. Retrieve older relevant turns, file chunks, and artifacts using lexical search first and embeddings when justified.
5. Add a versioned conversation summary that records decisions, constraints, open tasks, and artifact references.
6. Convert canonical messages and tools into the target provider's current request format.
7. Record a context snapshot hash, token estimate, included source IDs, and summary version for auditability.



Illustrative allocation only. Recalculate against each model's context window and current task.

Critical rule

Never send all rejected candidate responses during an ordinary switch. They can contradict the selected path, increase cost, and confuse the target model. Include them only when the user explicitly compares, quotes, or activates them.

6.2 Context compression strategy

Layer	Stored form	When included
Recent turns	Verbatim canonical messages	Always, within a recency budget.
Pinned facts	User or workspace-approved memory	When relevant to the current project and permitted.
Running summary	Structured decisions, constraints, tasks, entities	When history is longer than the target context budget.
Retrieved history	Top relevant messages plus provenance IDs	For older facts or reopened tasks.
Files	Metadata, extracted chunks, thumbnails, signed object reference	Only if the target provider supports the content and the user has access.
Artifacts	Code, image, document, or tool result references	When the current task depends on them.

A summary is not treated as truth simply because an LLM wrote it. It should be regenerated from canonical records, versioned, and invalidated when the user changes branches or edits earlier content. Important facts can be pinned by the user rather than inferred permanently.

7. Data model and transactional integrity

PostgreSQL is the source of truth for history, branches, usage, and credit reconciliation.

Entity	Key fields	Purpose
users	id, email, name, status, created_at	Identity and account lifecycle.
workspaces	id, owner_id, plan, retention_policy	Future team boundary and policy scope.
conversations	id, workspace_id, title, active_head_id, mode	Top-level chat and active branch pointer.
turns	id, conversation_id, parent_response_id, user_content, created_at	One user input anchored to the selected prior response.
model_runs	id, turn_id, provider, model_key, status, request_group_id	Lifecycle of one external model request.
model_responses	id, run_id, content, selected_at, finish_reason	Persistent candidate response and selection state.
context_snapshots	id, run_id, summary_version, source_ids, token_estimate	Reproducibility of provider handoff.
files	id, workspace_id, object_key, mime, size, scan_status	Metadata and permission boundary for object storage.
artifacts	id, conversation_id, type, object_key, source_run_id	Generated reusable outputs.
usage_events	id, run_id, provider_usage, price_snapshot, actual_cost	Immutable normalized metering.
credit_transactions	id, user_id, request_group_id, amount, type, status	Ledger for grants, reserves, charges, releases, and refunds.
provider_registry	provider, model_key, capabilities, pricing_version, enabled	Dynamic model configuration and safe rollouts.

7.1 Credit ledger, not a mutable counter

```
BEGIN;
  SELECT available_credits FROM credit_wallets
  WHERE user_id = $1 FOR UPDATE;

  INSERT INTO credit_transactions
    (user_id, request_group_id, amount, type, status)
  VALUES ($1, $2, -$3, 'RESERVATION', 'PENDING');

  UPDATE credit_wallets
  SET available_credits = available_credits - $3,
      reserved_credits = reserved_credits + $3
  WHERE user_id = $1;
COMMIT;
```

After provider completion, the system writes actual usage and reconciles the reservation. This prevents double spending across parallel requests, supports refunds, and gives support teams an auditable history. Money values use integer minor units or precise decimals; never binary floating point.

7.2 Essential constraints and indexes

- Unique idempotency key per user and logical request group.
- At most one active selected response per turn, enforced transactionally.
- Foreign keys from runs to turns and responses to runs; soft deletion only where retention policy requires it.
- Indexes on conversation_id + created_at, turn_id + provider, user_id + created_at, and request_group_id.
- JSONB only for provider-specific metadata and versioned capability snapshots, not for all relational state.

8. API and streaming contract

Commands use REST; long-running provider output uses server-sent events.

Endpoint	Method	Purpose
/v1/conversations	POST	Create a conversation with default mode and provider selection.
/v1/conversations/:id/turns	POST	Create one user turn and one or more model runs.
/v1/request-groups/:id/events	GET (SSE)	Multiplex provider events; reconnect using Last-Event-ID.
/v1/turns/:id/select	POST	Activate a response candidate and move the conversation head.
/v1/conversations/:id/switch	POST	Validate target provider and create the next model run with a context snapshot.
/v1/model-runs/:id/cancel	POST	Cancel one provider without canceling the entire comparison.
/v1/files	POST	Create signed upload intent, then scan and process asynchronously.
/v1/usage	GET	Return wallet, estimates, ledger, and actual usage summaries.

8.1 Example normalized SSE events

```
id: 1842
event: content.delta
data: {"runId":"run_gpt_01","provider":"openai","delta":"Use a modular..."}

id: 1843
event: run.status
data: {"runId":"run_claude_01","status":"completed","latencyMs":4210}

id: 1844
event: run.error
data: {"runId":"run_gemini_01","code":"RATE_LIMITED","retryAfterMs":8000}
```

SSE is the V1 default because the browser receives mostly one-way token and status streams. Use WebSockets later only for genuinely bidirectional, low-latency sessions such as live voice or continuous tool interaction.

9. Recommended market-aligned technology stack

Current stable technologies, chosen for employability, delivery speed, and a credible scaling path.

Layer	Primary recommendation	Rationale and alternative
Language	TypeScript end-to-end	Shared contracts and fast full-stack development. Generate schemas; do not share database models directly with the UI.
Web	Next.js 16 App Router, React 19, Tailwind CSS, shadcn/ui	Excellent streaming UX and deployment ecosystem. Vite + React is simpler if SEO/server rendering is unnecessary.
State	TanStack Query + small local store	Server state stays cached and invalidated predictably; avoid putting all chat data into a global client store.
API	NestJS 11 + Fastify on Node.js 24 LTS	Strong modules, DI, validation, OpenAPI, queues, SSE, and tests. Fastify lowers overhead while Nest keeps structure.
ORM	Prisma with explicit SQL migrations	Good type safety and developer experience. Use raw SQL for complex indexes, locks, JSONB, and performance-critical queries.
Database	Managed PostgreSQL 18	Transactions and relational integrity fit branching and credits. Use JSONB selectively and pgvector only when semantic retrieval is needed.
Cache / coordination	Managed Redis	Rate limits, short-lived stream state, distributed locks, cache, and BullMQ jobs. PostgreSQL remains source of truth.
Files	S3-compatible object storage	Signed uploads, lifecycle rules, inexpensive large objects, and CDN integration.
Transport	REST + SSE	Straightforward commands plus independent streaming provider runs. Add WebSockets only for bidirectional real-time modes.
Testing	Vitest/Jest, Supertest, Playwright, Testcontainers	Unit, API, UI, and real dependency integration coverage.
Observability	OpenTelemetry + structured logs + error tracking	Provider latency, token use, trace correlation, and cost need a shared trace model.
CI/CD	GitHub Actions + Docker + IaC	Repeatable builds, tests, scans, previews, migrations, and deployments.

Version policy

Pin exact production versions in lockfiles and container images, use an automated dependency update tool, and review provider SDK/API migrations monthly. Model IDs and capability data belong in configuration, not in source-code conditionals.

9.1 Why PostgreSQL rather than MongoDB

The core records contain strong relationships and money-like invariants: users own workspaces, conversations contain turns, turns contain multiple runs, one response becomes active, and usage reconciles against ledger transactions. PostgreSQL transactions, constraints, joins, JSONB, full-text search, and optional vector support handle this combination without introducing a second primary database.

9.2 Why NestJS rather than only Next.js route handlers

Next.js route handlers can ship a prototype, but OmniRoute's backend has provider adapters, background jobs, ledger transactions, context policies, file processing, retries, and observability. A separate NestJS application makes those responsibilities explicit and gives the project stronger backend engineering depth. Keep the web application focused on experience and the API focused on orchestration.

10. Building structure and repository design

A monorepo keeps contracts synchronized while preserving deployable boundaries.

```
omniroute/  
  apps/  
    web/           # Next.js UI  
    api/           # NestJS HTTP + SSE API  
    worker/        # BullMQ jobs: summaries, files, analytics  
  packages/  
    contracts/     # Zod schemas, API DTOs, event types  
    provider-core/ # Canonical request/response interfaces  
    provider-openai/  
    provider-anthropic/  
    provider-gemini/  
    ui/            # Shared design system  
    observability/ # Logging, tracing, metrics helpers  
    config/        # Typed environment configuration  
  infra/  
    docker/  
    terraform/     # Add when deploying managed cloud resources  
  docs/  
    architecture-decisions/  
    api/  
    runbooks/  
  .github/workflows/  
  docker-compose.yml # Local Postgres, Redis, object-store emulator  
  pnpm-workspace.yaml  
  turbo.json
```

10.1 NestJS module layout

Module	Internal components
conversations	controllers, branch service, turn repository, selection policy
context	assembler, summarizer, retriever, token budgeter, format converters
providers	registry, orchestrator, adapters, retry policy, circuit breakers
usage	estimator, reservation service, reconciler, pricing snapshots, ledger repository
files	upload intents, scanner, extractor, chunker, artifact links
streaming	SSE multiplexer, event store, reconnect and terminal-state logic
policy	rate limits, capability checks, tool permissions, retention

Keep repository interfaces inside each module and provider SDK imports inside adapter packages. This allows unit tests to substitute fake providers and prevents provider response types from leaking into the database or UI.

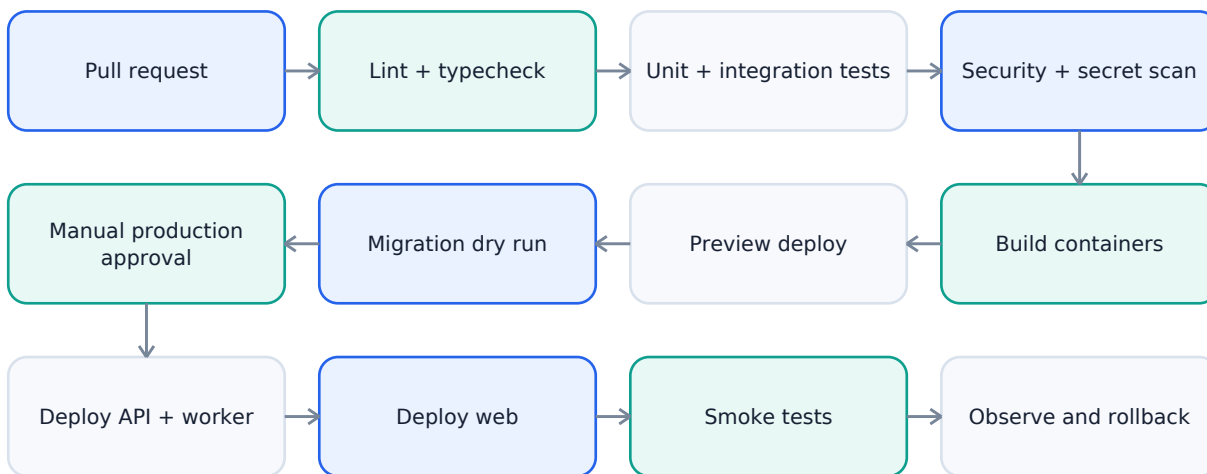
11. DevOps, deployment, and environments

Build a professional delivery path without operating infrastructure you do not yet need.

11.1 Recommended environments

Environment	Purpose	Data rule
Local	Docker Compose dependencies and provider mocks	Synthetic fixtures; developer-owned API keys only in secret storage.
Preview	Per-pull-request web and API validation	No production data; short retention; low provider spend caps.
Staging	Migration rehearsal, load tests, provider smoke tests	Production-like schema with sanitized data.
Production	Real users and billing	Encrypted, audited, backed up, and retention-controlled.

11.2 CI/CD pipeline



- Use multi-stage Docker builds, non-root runtime users, health endpoints, and immutable image tags.
- Run database migrations as a controlled release step. Use expand-migrate-contract for breaking schema changes.
- Keep secrets in the deployment platform or cloud secret manager, never in repository or client bundles.
- Back up PostgreSQL and test restoration. Object storage needs lifecycle and deletion policies.
- Do not deploy Kubernetes until container count, traffic variability, compliance, or team operations justify it.

11.3 Deployment path

Stage	Web	API / worker	Data
MVP	Vercel	Railway, Render, Fly.io, or managed container service	Managed Postgres + managed Redis + S3-compatible storage
Growth	Vercel or CDN-hosted Next.js	AWS ECS/Fargate or equivalent	RDS Postgres, managed Redis, S3, queue
Large scale	Multi-region edge delivery	Kubernetes or mature container platform if justified	Read replicas, partitioning, regional storage and DR

12. Security, privacy, and responsible AI controls

A multi-provider product expands the data boundary; security must be visible in the architecture.

Risk	Control
Prompt injection through files or retrieved content	Treat external text as untrusted data; isolate instructions; allowlist tools; require approval for side effects.
Sensitive data sent to multiple providers	Clear provider selection, privacy notice, workspace policy, optional redaction, and automatic fallback opt-in.
Provider keys exposed	Server-side secrets only, separate keys per environment, rotation, spend limits, and no key material in logs.
Cross-tenant access	Workspace-scoped authorization on every query and object key; signed URLs with short expiry; deny by default.
Improper model output handling	Escape rendered HTML, sanitize Markdown, sandbox code, validate tool arguments, and never execute model text directly.
Unbounded consumption	Rate limits, per-user budgets, max output tokens, file limits, concurrency caps, reservations, and circuit breakers.
Data retention mismatch	Document provider retention terms, define application retention, support export and deletion, and record deletion state.
Supply-chain compromise	Lockfiles, signed builds, dependency scanning, SBOM, minimal images, and automated patch workflow.

12.1 Minimum production security checklist

- HttpOnly, Secure, SameSite cookies; CSRF protection where needed; short session lifetime with rotation.
- Schema validation at every API boundary and strict content-type and size checks.
- Per-workspace authorization before database reads, file access, provider calls, and exports.
- Audit events for login, file access, provider switch, tool execution, credit adjustment, export, and deletion.
- Prompt and response content excluded from routine logs; use IDs, counts, hashes, and redacted metadata.
- Threat model reviewed against OWASP risks including prompt injection, sensitive disclosure, improper output handling, excessive agency, and unbounded consumption.

Tool execution boundary

The model proposes a tool call; OmniRoute validates the schema, authorization, scope, and required user approval; the application executes it; the result returns to the model. Never grant a model direct database or cloud credentials.

13. Observability, reliability, and quality evaluation

A successful response is more than HTTP 200; it must be timely, complete, attributable, and economically recorded.

Signal	Key measures
User experience	Time to first token, time to complete, comparison completion, switch time, reconnect success
Provider	Success rate, timeout rate, 429 rate, input/output tokens, latency by model and region
Product	Response selection rate, provider selection share by task, switches per conversation, branch reactivation
Economics	Actual provider cost, reserved vs reconciled credits, gross margin per mode, free-credit burn
Context	Context token size, compression ratio, retrieval hit rate, switch coherence feedback
System	API error rate, queue depth, database latency, Redis health, object processing failures

13.1 Suggested service objectives for early production

Objective	Initial target	Action when missed
API availability excluding providers	99.9% monthly	Inspect traces, rollback release, fail closed on credit uncertainty.
SSE reconnect recovery	> 99% within retained event window	Increase event retention and test Last-Event-ID behavior.
Credit reconciliation	100% of terminal runs	Alert on aged reservations; run idempotent reconciler.
Context switch preparation	p95 under 2 seconds before provider time	Profile retrieval and summary generation; cache safe snapshots.
Provider run completion	Measured separately per model	Do not hide provider instability inside aggregate API uptime.

13.2 Quality and routing dataset

The user's selected response is a valuable preference signal, but it is not automatically a universal quality label. Selection can be influenced by provider order, response length, latency, brand, formatting, or familiarity. Randomize tab order during experiments, capture task category and latency, and separate explicit selection from passive viewing before training a future router.

- Track provider shown, position, completion status, response length, latency, selected response, later switch, regenerate, and thumbs feedback.
- Build small expert-reviewed evaluation sets before enabling Auto Pick, and retain derived metrics longer than raw content when privacy permits.

14. Implementation roadmap

Ship the complete core loop first, then deepen reliability and intelligence.

14.1 Focused seven-day demonstrable MVP

Day	Build	Exit criterion
1	Monorepo, Next.js shell, NestJS API, Google login, local Postgres/Redis	Authenticated user can create and reopen a conversation.
2	Canonical provider interface and OpenAI/Anthropic/Gemini adapters	Fake and live adapters pass the same contract tests.
3	Parallel run group, SSE multiplexer, provider tabs, cancellation	Three independent responses stream into one comparison view.
4	Turns, model runs, responses, selection, active branch	User selects a response and continues with its provider.
5	Context assembler, provider switch, recent history + summary	A second provider continues the selected conversation coherently.
6	Credit reservations, usage reconciliation, limits, partial failure UX	Parallel requests cannot double-spend and failures refund correctly.
7	Tests, security checks, Docker, CI/CD, deployment, README and demo	Public demo works across desktop/mobile with observability and smoke tests.

Reality check

Seven days can produce a strong portfolio MVP only if scope stays narrow and provider/tool edge cases are deliberately deferred. A trustworthy commercial release needs additional work on retention, abuse, billing, evaluation, accessibility, and operations.

14.2 Six-week production-minded extension

Phase	Additions
Weeks 2-3	Robust reconnect, file lifecycle, summary versioning, provider registry UI, admin spend controls, accessibility, export/delete.
Weeks 4-5	Load tests, prompt and output safety, team workspaces, analytics, cost dashboards, runbooks, backup/restore drills.
Week 6	Closed beta, task-specific evals, onboarding experiments, pricing trial, provider failure drills, support workflow.

14.3 Later, only after evidence

- BYOK or official provider account connections, implemented as credential types rather than API-key-only fields.
- Response comparison summaries and model recommendations.
- Personalized Auto Pick trained on controlled selection data.
- Workspaces, RAG, reusable agents, and domain-specific tool packs.

15. Testing and acceptance strategy

Provider APIs are nondeterministic; contracts and invariants must be deterministic.

Test layer	What to test
Unit	Context budgeting, branch resolution, credit arithmetic, capability checks, error normalization, retry policy.
Adapter contract	Recorded/fake provider streams, token usage mapping, cancellation, tool-call normalization, malformed events.
Database integration	Transactions, row locks, unique idempotency keys, selection constraints, reservation reconciliation.
API integration	Auth, authorization, POST idempotency, SSE order, reconnect, cancellation, partial provider failure.
End-to-end	Login, Compare 3, select, continue, switch, upload, low credits, provider failure, mobile navigation.
Load	Concurrent comparisons, long streams, Redis outage, slow provider, database pool saturation, queue backpressure.
Security	Cross-tenant access, prompt injection fixtures, HTML/Markdown sanitization, file bombs, secret leakage, rate limits.
Evaluation	Branch coherence after switching, context fact retention, task success, user preference bias by tab position.

15.1 MVP acceptance checklist

- A single prompt creates exactly one turn and three uniquely identifiable runs.
- Each provider can stream, complete, fail, retry, or cancel without corrupting the other runs.
- Selecting one response moves the active head exactly once and survives refresh.
- Switching providers sends the active selected path, not all candidate responses.
- Credits cannot go negative because of parallel sends; every terminal run is reconciled.
- Files cannot be accessed across users/workspaces and are not exposed through permanent public URLs.
- The UI works on mobile, keyboard navigation is usable, and loading/error states are clear.
- Logs and traces correlate user turn, request group, model run, provider request, and ledger entries without logging raw secrets.

16. Product positives, negatives, and market reality

The idea is valuable, but its cost structure and defensibility must be treated honestly.

16.1 Positive characteristics

Positive	Why it is meaningful
Immediate user value	Removes copying prompts, context, files, and failed attempts between provider websites.
Human-in-the-loop quality	Users choose the preferred response before a later router tries to predict preference.
Provider resilience	Work can continue when a provider is limited, slow, or unsuitable for a capability.
Compounding data asset	Selections, switches, task categories, latency, and outcomes can improve future recommendations.
Strong technical portfolio	Demonstrates concurrency, streaming, adapters, transactional credits, branching, DevOps, and observability.
Expandable platform	Files, workspaces, tools, RAG, agents, and more providers can attach to the same conversation layer.
Clear message	'Your conversation stays. Your AI can change' is simple to understand and demonstrate.

16.2 Negative characteristics and mitigations

Negative / risk	Impact	Mitigation
Compare mode costs roughly multiple provider calls	Free usage can become expensive and margins thin	Make Single AI default after selection, show estimates, cap outputs, use reservations, and price Compare mode honestly.
Slowest provider shapes perceived latency	Comparison may feel slower than a normal chat	Stream independently, allow early selection, show per-tab progress, and set provider timeouts.
Context translation is imperfect	New provider may misunderstand summaries or provider-specific tool outputs	Use canonical history, provenance, typed artifacts, recent verbatim turns, and explicit context snapshots.
Capabilities and APIs change frequently	Adapters break and static tables become misleading	Version adapters, contract-test them, maintain a provider registry, and run scheduled smoke tests.
Privacy surface expands	One prompt can reach several external processors	Clear consent, selected providers, retention controls, redaction, and no silent cross-provider fallback.
Aggregator features are copyable	Large platforms can add model tabs	Differentiate through persistent cross-model projects, excellent switching, team workflows, evaluations, and vertical use cases.
Comparison can create decision fatigue	Users may spend longer reading three similar answers	Use Compare only for high-value tasks, show concise previews, and add evidence-based recommendation later.
Output quality remains nondeterministic	Three confident wrong answers are still wrong	Citations, tools, evals, warnings, user verification, and task-specific checks.
Credit systems can confuse users	Unexpected deductions damage trust	Estimate, reserve, reconcile, display actual usage, and keep an auditable ledger.
Vendor concentration	Provider policy, pricing, or availability can change	Adapter independence, spend caps, model registry, provider diversity, and exit-ready data export.

16.3 Commercial and strategic assessment

The concept is attractive as a productivity product and excellent as a resume project. As a standalone horizontal startup, however, 'access many models' is not enough. The strongest strategic path is to own the workflow and state around model use: projects, branches, files, reusable instructions, team approvals, evaluations, and provider-independent audit history.

Question	Assessment
Is there a real problem?	Yes. Users manually move prompts, files, and context across tools when a model fails or another capability is needed.
Is the MVP feasible?	Yes. Three provider adapters, parallel streaming, branching, context handoff, credits, and history are achievable with disciplined scope.
Is it cheap to operate?	No. Compare mode multiplies inference calls, so quotas, pricing, caching, and output controls are core product design.
Is it defensible today?	Only moderately. Defensibility grows from workflow depth, team adoption, evaluation data, and domain specialization.
Best initial positioning	A developer and technical-learning workspace for comparing solutions and continuing across models.
Go / no-go	Go for portfolio and closed-beta validation. Do not scale free credits or infrastructure before measuring retention and willingness to pay.

Best business model sequence

Begin with limited free platform credits and a paid usage tier. Add team workspaces and administrative controls before complicated autonomous agents. Consider BYOK only after the core experience is trusted and secure.

17. Final recommendation

Build the conversation layer first and let model intelligence remain replaceable.

OmniRoute should launch around one polished loop: ask three, choose one, continue, and switch without re-explaining. Everything in the architecture should protect that loop. The frontend makes comparison calm and switching obvious. The conversation engine owns the active branch. The context engine transfers relevant state. Provider adapters absorb API differences. PostgreSQL protects history and credits. Redis coordinates short-lived work. Observability connects user experience, reliability, and cost.

The product becomes weaker when it attempts every provider feature, autonomous routing, and infrastructure pattern at once. It becomes stronger when it is transparent about costs, explicit about provider boundaries, careful with privacy, and excellent at preserving work across provider changes.

Build order in one line

Reliable provider adapters -> parallel streaming -> response selection -> active branch -> context handoff -> ledger accounting -> files and capabilities -> evaluation data -> smart recommendations.

**YOUR CONVERSATION STAYS.
YOUR AI CAN CHANGE.**

Sources and current-technology notes

Official documentation checked for this September 2026 recommendation.

Source	How it informed the report
Next.js documentation	Latest documentation listed Next.js 16.3.4 and describes the App Router, streaming, authentication, testing, and deployment features.
NestJS documentation	NestJS documents TypeScript architecture, Fastify support, SSE, queues, security, microservices, testing, OpenAPI, and observability.
Node.js release schedule	Node.js 24 was the latest LTS line on the checked page; production applications are advised to use Active or Maintenance LTS.
OpenAI Responses API migration guidance	OpenAI states the Assistants API was sunset on August 26, 2026 and directs new integrations to the Responses API.
Anthropic Messages streaming	Anthropic documents incremental Messages API output using server-sent events.
Gemini Interactions API	Google describes Interactions as generally available and recommended for new Gemini projects as of June 2026.
PostgreSQL current documentation	Current PostgreSQL documentation; JSONB supports efficient processing and indexing, while standard relational features provide transactional integrity.
pgvector	The official project documents exact and approximate vector similarity search inside PostgreSQL.
Redis documentation	Redis documents caching, streams, real-time context, search, and production deployment options.
GitHub Actions	GitHub describes repository-native automation for customized workflows including CI/CD.
Docker overview	Docker's official overview describes containerized application packaging and execution.
OpenTelemetry	Official documentation for traces, metrics, logs, instrumentation, collectors, and context propagation.
OWASP GenAI Top 10	OWASP's 2025 list includes prompt injection, sensitive disclosure, improper output handling, excessive agency, and unbounded consumption.

Technology versions and provider capabilities change quickly. Re-check stable releases, pricing, quotas, data retention, and API migration notices immediately before implementation and again before launch. The report avoids fixed provider price figures for that reason.

End of report